

PERIYAR MANIAMMAI
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)
Established Under Sec. 3 of UGC Act, 1956 • NAAC Accredited
think • innovate • transform

CASE STUDY REPORT

TOPIC: HOSPITAL READMISSION RECORDS – DATA QUALITY & WRANGLING IN HEALTHCARE

P24CS9XX

PROFESSIONAL ELECTIVE -I
DATA SCIENCE

T. GOBIKA
Reg. No: 125262006
M.Tech CSE

SUBMITTED TO
Mrs. Dr. M. Prabha
Associate Professor CSE

HOSPITAL READMISSION RECORDS – DATA QUALITY & WRANGLING IN HEALTHCARE

ABSTRACT

Healthcare organizations generate large volumes of patient information through electronic health records, hospital admission systems, laboratory reports, and treatment records. However, these records frequently contain missing values, inconsistent categories, duplicate entries, incorrect data formats, and abnormal numerical values. Such data-quality problems can affect the accuracy of healthcare analytics and decision-making.

This project, titled **"Hospital Readmission Records – Data Quality & Wrangling in Healthcare,"** focuses on identifying, analysing, and correcting data-quality issues in hospital readmission records using Python-based data-wrangling techniques.

The project uses the publicly available Diabetes 130-US Hospitals dataset from the UCI Machine Learning Repository. The dataset contains approximately 100,000 hospital encounters collected from 130 US hospitals during the period 1999–2008.

The proposed system performs data profiling, missing-value identification, outlier detection, categorical-value standardization, numerical normalization, and integration with an ICD-9 diagnosis lookup table. It also transforms encounter-level records into a patient-level dataset using suitable aggregation techniques.

Python libraries such as Pandas and NumPy are used for data manipulation and statistical analysis, while Jupyter Notebook provides an interactive environment for implementation and documentation.

The final outcome is a cleaned and structured healthcare dataset, a reproducible data-cleaning notebook, and a data-quality report documenting all cleaning decisions. This work provides a reliable foundation for further healthcare analytics and hospital readmission prediction, supporting **United Nations Sustainable Development Goal 3 – Good Health and Well-being**.

Keywords: Healthcare Analytics, Data Quality, Data Wrangling, Hospital Readmission, Missing Values, Outlier Detection, ICD-9, Pandas, NumPy.

CHAPTER 1: INTRODUCTION

1.1 Background

Healthcare data management is an important component of modern medical information systems. Hospitals collect patient information related to admission, discharge, diagnosis, medication, laboratory tests, and treatment procedures.

Although electronic health records provide valuable information, healthcare datasets often contain quality-related problems, including:

- Missing or incomplete patient information.
- Duplicate hospital encounter records.
- Inconsistent diagnosis categories.
- Incorrect numerical values.
- Different formats for categorical variables.
- Inconsistent medical codes.
- Repeated patient records across multiple hospital visits.

These issues make it difficult to perform accurate data analysis.

Data wrangling is the process of converting raw and unstructured data into a consistent, clean, and analysis-ready format.

This project applies systematic data-quality assessment and data-wrangling techniques to hospital readmission records.

1.2 Problem Statement

The Diabetes 130-US Hospitals dataset contains approximately 100,000 hospital encounter records. However, the raw dataset may contain missing values represented by special symbols, inconsistent categorical values, numerical outliers, and multiple records belonging to the same patient.

Using this dataset without proper preprocessing may produce misleading statistical results and reduce the reliability of subsequent healthcare analysis.

Therefore, this project aims to develop a systematic Python-based data-wrangling workflow to identify and resolve data-quality issues, normalize numerical attributes, integrate diagnosis information using ICD-9 codes, and transform encounter-level records into a patient-level dataset.

1.3 Objectives

The major objectives are:

1. To understand and analyse the structure of hospital readmission records.
2. To perform data profiling and identify data-quality problems.
3. To detect and handle missing values represented by '?' symbols.
4. To identify and treat abnormal numerical values and outliers.
5. To standardize inconsistent categorical values.
6. To normalize numerical attributes for consistent analysis.
7. To integrate hospital diagnosis records with an ICD-9 lookup table.

8. To transform multiple hospital encounters into one consolidated record per patient.
9. To document every data-cleaning decision.
10. To generate a clean dataset suitable for further healthcare analytics.

1.4 Scope of the Project

The project concentrates on data quality and data preprocessing rather than developing a complete readmission prediction model.

The proposed workflow covers:

- Raw healthcare dataset inspection.
- Data-quality profiling.
- Missing-value treatment.
- Duplicate detection.
- Outlier analysis.
- Categorical standardization.
- Numerical normalization.
- ICD-9 diagnosis integration.
- Patient-level aggregation & Data-quality reporting

The resulting dataset can be used in future research involving readmission prediction, patient risk analysis, and healthcare resource management.

CHAPTER 2: DATASET DESCRIPTION

2.1 Dataset Overview

The project uses the **Diabetes 130-US Hospitals Dataset**, available through the UCI Machine Learning Repository.

Dataset characteristics:

Attribute	**Description**
Dataset name	Diabetes 130-US Hospitals
Domain	Healthcare
Source	UCI Machine Learning Repository
Number of encounters	Approximately 101,766
Number of hospitals	130

Number of attributes	50
Collection period	1999–2008
Main subject	Hospital encounters involving diabetic patients
Target attribute	Readmitted
File format	CSV

2.2 Important Dataset Attributes

Attribute	**Description**
encounter_id	Unique hospital encounter identifier
patient_nbr	Patient identification number
Race	Patient race category
Gender	Patient gender
Age	Patient age group
admission_type_id	Type of hospital admission
discharge_disposition_id	Patient discharge category
admission_source_id	Source of hospital admission
time_in_hospital	Number of days in hospital
num_lab_procedures	Number of laboratory procedures
num_procedures	Number of medical procedures
num_medications	Number of medications
number_outpatient	Previous outpatient visits
number_emergency	Previous emergency visits
number_inpatient	Previous inpatient visits
diag_1	Primary diagnosis code
diag_2	Secondary diagnosis code
diag_3	Additional diagnosis code
diabetesMed	Whether diabetes medication was prescribed
Readmitted	Readmission status

2.3 Readmission Categories

The dataset contains three important readmission categories:

- <30: Readmitted within 30 days.
- >30: Readmitted after 30 days.
- NO: No recorded readmission during the specified observation framework.

These categories must be retained and interpreted consistently during preprocessing.

2.4 Dataset Source

UCI Machine Learning Repository:

https://archive.ics.uci.edu/dataset/296/diabetes+130+us+hospitals+for+years+1999+2008

The dataset and its documentation should be downloaded from the official repository.

CHAPTER 3: PROPOSED METHODOLOGY

3.1 System Architecture

The proposed project follows a systematic data-wrangling pipeline.

Workflow:

1. Dataset collection.
2. Data loading.
3. Data profiling.
4. Missing-value analysis.
5. Duplicate detection.
6. Outlier detection and treatment.
7. Categorical standardization.
8. Numerical normalization.
9. ICD-9 lookup integration.
10. Patient-level data transformation.
11. Data-quality validation.
12. Export of cleaned dataset and report.

3.2 Data Profiling

Data profiling is the first stage of the proposed system.

It helps understand the dataset's structure, completeness, uniqueness, and statistical properties.

The following information is examined:

- Number of rows and columns.
- Data types of attributes.
- Number of unique values.
- Missing-value percentage.
- Duplicate records.
- Minimum and maximum numerical values.
- Mean, median, and standard deviation.

The output of this stage is a data-quality profile.

3.3 Missing-Value Handling

Missing values in the dataset are represented using the special character ? in several categorical columns.

The system converts these symbols into standard missing values.

Treatment strategy:

- Identify missing values in every column.
- Calculate missing-value percentages.
- Remove or retain columns depending on their analytical importance.
- Use suitable imputation for selected attributes.
- Preserve missingness indicators where useful.
- Avoid blindly replacing missing diagnosis codes with an arbitrary diagnosis.

For example, missing categorical values can be represented as Unknown when this is appropriate for the intended analysis.

3.4 Outlier Detection

Outliers are numerical observations that differ significantly from the general distribution of a variable.

This project uses:

Interquartile Range (IQR) Method

[IQR=Q3-Q1]

Lower boundary:

[LB=Q1-1.5(IQR)]

Upper boundary:

[UB=Q3+1.5(IQR)]

Values outside these boundaries are flagged for investigation.

However, healthcare values should not be removed automatically. Some unusual observations may represent genuine clinical conditions or uncommon patient histories.

The proposed system therefore distinguishes between:

- Invalid values.
- Clinically implausible values.
- Statistically unusual but valid observations.

Appropriate treatment may include correction, capping, retention, or exclusion with justification.

3.5 Categorical Data Standardization

Categorical attributes may contain inconsistent values, spelling variations, and different representations.

The system standardizes categorical values using:

- Whitespace removal.
- Consistent letter casing where appropriate.
- Mapping of equivalent category labels.
- Consistent missing-value representation.
- Validation against permitted categories.

For example: Male, male, and MALE can be standardized to Male.

The original values should be retained in the audit record when transformations are performed.

3.6 Numerical Normalization

Numerical normalization converts numerical attributes into a comparable scale.

This project uses Min-Max normalization for selected numerical attributes.

[X_{normalized}=\frac{X-X_{min}}{X_{max}-X_{min}}]

The normalized values generally fall between 0 and 1.

Selected variables may include:

- Time in hospital.
- Number of laboratory procedures.
- Number of medical procedures.
- Number of medications.
- Outpatient visit count.
- Emergency visit count.
- Inpatient visit count.

Normalization should be applied after invalid values and outliers have been investigated.

The original numerical values must also be preserved for clinical interpretation.

3.7 ICD-9 Lookup Table Integration

ICD-9 is a medical classification system used to represent diagnoses.

The dataset contains diagnosis attributes:

- diag_1
- diag_2
- diag_3

An external ICD-9 lookup table is integrated to obtain meaningful diagnosis descriptions.

Integration procedure:

1. Obtain a suitable ICD-9 diagnosis reference table.
2. Load the lookup table into Pandas.
3. Standardize diagnosis codes in both datasets.
4. Merge diagnosis records with the lookup table.
5. Check unmatched diagnosis codes.
6. Preserve original diagnosis codes and descriptions.

Example:

Diagnosis code	**Description**
250.00	Diabetes mellitus without mention of complication
401.9	Unspecified essential hypertension
414.01	Coronary atherosclerosis of native coronary artery

The descriptions are illustrative; the selected lookup table must be checked for the exact code descriptions and coding version.

3.8 Patient-Level Data Transformation

The original dataset contains multiple hospital encounters for some patients.

Therefore, the project transforms encounter-level records into one consolidated record per patient.

The patient identifier patient_nbr is used as the grouping attribute.

Possible aggregation techniques:

Attribute	**Aggregation method**
patient_nbr	Grouping identifier
encounter_id	Count of encounters
time_in_hospital	Mean, maximum, or total
num_lab_procedures	Mean or total
num_medications	Mean or maximum
number_outpatient	Latest or cumulative value, based on definition
number_emergency	Latest or cumulative value, based on definition
number_inpatient	Latest or cumulative value, based on definition
Readmitted	Derived patient-level outcome

A patient-level readmission outcome must have an explicitly defined observation period and index encounter.

For example, the project can define whether the patient was readmitted within 30 days after an eligible index encounter.

The original encounter-level dataset should be preserved because aggregation can remove temporal information.

3.9 Documentation of Cleaning Decisions

Every transformation is documented in a data-cleaning log.

The log contains:

- Attribute name.
- Original problem.
- Cleaning method.
- Reason for selecting the method.

- Number of affected records.
- Validation result.

This ensures transparency, reproducibility, and accountability in healthcare data processing.

CHAPTER 4: IMPLEMENTATION

4.1 Software and Hardware Requirements

Software requirements

Software / Library	Purpose
Python 3.x	Programming language
Jupyter Notebook	Interactive development
Pandas	Data manipulation
NumPy	Numerical operations
Matplotlib	Data visualization
Seaborn	Statistical visualization

Hardware requirements

- Processor: Intel Core i3 or above.
- RAM: Minimum 8 GB recommended.
- Storage: Minimum 2 GB free space for dataset and processing.
- Operating System: Windows, Linux, or macOS.

4.2 Python Implementation

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Dataset

```
df = pd.read_csv("diabetic_data.csv")
print("Dataset Shape:", df.shape)
print(df.head())
```

Step 3: Data Profiling

```
print(df.info())
print(df.describe(include="all"))
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:")
print(df.duplicated().sum())
```

Step 4: Convert '?' into Missing Values

```
df.replace("?", np.nan, inplace=True)
missing = df.isnull().sum()
print(missing[missing > 0])
```

Step 5: Missing-Value Percentage

```
missing_percentage = (
    df.isnull().mean() * 100
).sort_values(ascending=False)
print(missing_percentage)
```

Step 6: Remove Duplicate Encounters

```
df = df.drop_duplicates(
    subset=["encounter_id"]
)
print("Shape after duplicate removal:", df.shape)
```

The encounter identifier is used because each encounter should represent a unique hospital visit. Patient-level duplicates must not be removed at this stage.

Step 7: Standardize Categorical Values

```
categorical_columns = df.select_dtypes(
    include=["object"]
).columns
for col in categorical_columns:
    df[col] = df[col].str.strip()
```

```
print(df["gender"].value_counts(dropna=False))
```

Step 8: Outlier Detection Using IQR

```
column = "time_in_hospital"
Q1 = df[column].quantile(0.25)
Q3 = df[column].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[
    (df[column] < lower) |
    (df[column] > upper)
]
print("Number of outliers:", len(outliers))
```

The identified records are flagged for examination rather than automatically deleted.

Step 9: Numerical Normalization

```
numeric_columns = [
    "time_in_hospital",
    "num_lab_procedures",
    "num_procedures",
    "num_medications",
    "number_outpatient",
    "number_emergency",
    "number_inpatient"
]
df_normalized = df.copy()
for col in numeric_columns:
    minimum = df_normalized[col].min()
    maximum = df_normalized[col].max()
    if maximum != minimum:
```

```
    df_normalized[col] = (
        (df_normalized[col] - minimum) /
        (maximum - minimum)
    )
```

This produces a separate normalized dataset while preserving the original dataset.

Step 10: ICD-9 Lookup Integration

```
icd = pd.read_csv("icd9_lookup.csv")
icd["code"] = (
    icd["code"].astype(str).str.strip()
)
for col in ["diag_1", "diag_2", "diag_3"]:
    df[col] = df[col].astype("string").str.strip()
    df = df.merge(
        icd,
        left_on=col,
        right_on="code",
        how="left" )
```

For the final implementation, use separate diagnosis-description columns for each of the three diagnosis fields and verify that the merge does not change the original encounter count.

Step 11: Patient-Level Aggregation

```
patient_data = df.groupby(
    "patient_nbr"
).agg(
    total_encounters=("encounter_id", "count"),
    avg_hospital_days=("time_in_hospital", "mean"),
    avg_lab_procedures=("num_lab_procedures", "mean"),
    avg_medications=("num_medications", "mean")
).reset_index()
print(patient_data.head())
```

This is an initial patient-level summary. The final implementation should also derive readmission outcomes using chronological encounters and a clearly defined observation period.

Step 12: Export Cleaned Dataset

```
df.to_csv(
    "cleaned_hospital_records.csv",
    index=False
)
patient_data.to_csv(
    "patient_level_records.csv",
    index=False)
print("Cleaned datasets exported successfully.")
```

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Expected Results

The proposed project is expected to produce the following results:

1. A complete data-quality profile of the original hospital dataset.
2. Identification of missing values and their percentages.
3. Identification of duplicate encounter records.
4. Detection and documentation of numerical outliers.
5. Standardization of inconsistent categorical attributes.
6. Normalized numerical attributes.
7. Integration of ICD-9 diagnosis descriptions.
8. A consolidated patient-level dataset.
9. A detailed data-cleaning log.

5.2 Data Quality Evaluation

The quality of the cleaned dataset can be evaluated using the following dimensions:

Quality dimension	**Evaluation method**
Completeness	Percentage of non-missing values
Uniqueness	Duplicate encounter identification

Consistency	Standardized category verification
Validity	Range and permitted-value checks
Accuracy	Verification against trusted reference data
Integrity	ICD-9 lookup merge validation

5.3 Before and After Comparison

The following table is a template for recording actual experimental results.

Evaluation metric	**Original dataset**	**Cleaned dataset**
Total records	101,766 (expected source size)	To be measured
Missing values	To be measured	To be measured
Duplicate encounters	To be measured	To be measured
Inconsistent categories	To be measured	To be measured
Unmatched ICD-9 codes	Not evaluated	To be measured
Patient-level records	Encounter-level	To be measured

The final report should include actual values obtained from executing the notebook. No experimental improvement percentages should be claimed without measurement.

5.4 SDG 3 Contribution

This project supports **Sustainable Development Goal 3: Good Health and Well-being**.

Reliable healthcare information is essential for:

- Improving patient record management.
- Supporting healthcare research.
- Understanding hospital readmission patterns.
- Helping healthcare organizations analyse patient histories.
- Supporting evidence-based healthcare decisions.

The project contributes indirectly to better healthcare analytics by improving the quality of hospital data.

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENT

6.1 Conclusion

The project presents a systematic approach to healthcare data quality assessment and data wrangling using the Diabetes 130-US Hospitals dataset.

The proposed methodology identifies and addresses important data-quality problems, including missing values, duplicate encounters, inconsistent categorical attributes, numerical outliers, and diagnosis-code inconsistencies.

The integration of ICD-9 lookup information improves the interpretability of diagnosis records. Patient-level aggregation provides a consolidated representation of patients with multiple hospital encounters.

Python libraries such as Pandas and NumPy simplify data preprocessing and transformation, while Jupyter Notebook supports reproducible implementation and documentation.

The final cleaned dataset provides a foundation for further healthcare data analysis and supports the broader objective of improving healthcare information quality under SDG 3.

6.2 Future Enhancement

The project can be extended in the following ways:

1. Develop a machine-learning model to predict hospital readmission risk.
2. Apply advanced missing-value imputation techniques.
3. Integrate additional healthcare datasets.
4. Develop an interactive data-quality dashboard.
5. Perform patient risk analysis using clinical and admission attributes.
6. Apply explainable AI techniques to healthcare prediction.
7. Develop automated data-quality monitoring for hospital information systems.

REFERENCES

1. Strack, B., DeShazo, J. P., Gennings, C., et al. (2014). Impact of HbA1c Measurement on Hospital Readmission Rates: Analysis of 70,000 Clinical Database Patient Records. *BioMed Research International*, 2014.
2. UCI Machine Learning Repository. Diabetes 130-US Hospitals for Years 1999–2008. https://archive.ics.uci.edu/dataset/296/diabetes+130+us+hospitals+for+years+1999+2008
3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.

4. Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array Programming with NumPy. *Nature*, 585, 357–362.
5. World Health Organization. International Classification of Diseases (ICD). https://www.who.int/standards/classifications/classification-of-diseases
6. United Nations. Sustainable Development Goal 3: Good Health and Well-being. https://sdgs.un.org/goals/goal3